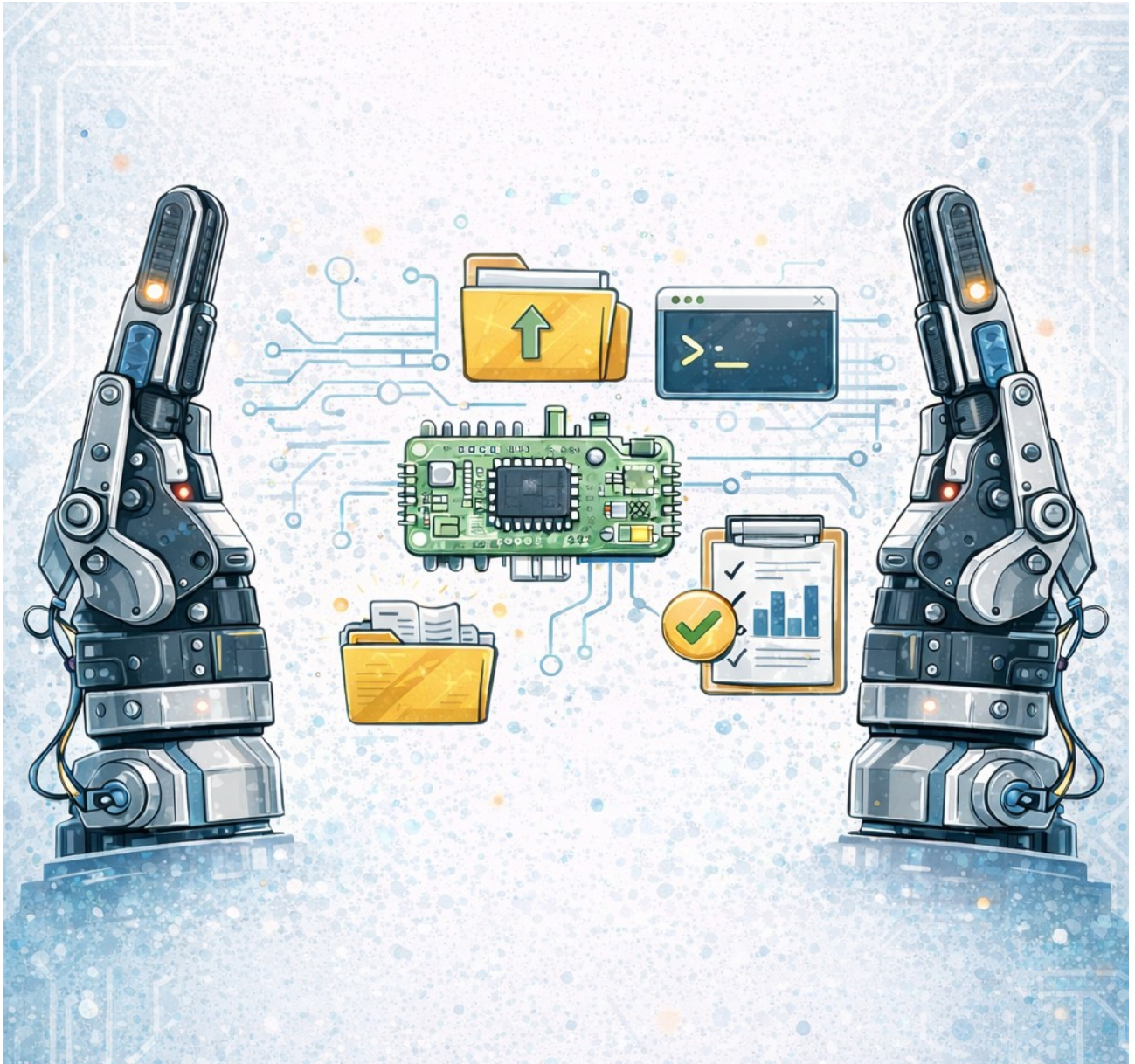


Matrices, Vectores y Archivos



Dimitry Company Cifre Repositorio: [UnInformaticoAburrido/Practicas SO 2025 26 at Practica2](#)

Nidhal Majoul Repositorio: <https://github.com/nidhal-majoul/Practicas SO 2025 26/tree/practica2>

1. Explicación de requisitos

La práctica pide crear un programa en C++ que lea un archivo con datos de sensores, guarde esa información en matrices y arrays, calcule la estabilidad del agarre y escriba los resultados en un archivo de salida. Además, después de compilarlo, hay que hacer un script en Bash que ejecute el programa, lo lance en segundo plano, espere a que termine y lo cierre correctamente si sigue activo.

Requisito	Cómo se cumple en la solución
Lectura del archivo	Se abre datos_pinza.txt con ifstream y se valida que el archivo exista antes de continuar.
Uso de matriz	Cada bloque de trabajo utiliza la matriz datos[100][3] para almacenar galga, fuerza izquierda y fuerza derecha.
Uso de arrays	Los valores también se separan en ids, galga, fuerza_izq y fuerza_der para procesarlos de forma más clara.
Cálculo de medias	Se suman todos los valores y al final se divide entre el número total de registros procesados.
Análisis de estabilidad	Se calcula la diferencia entre fuerza_izq y fuerza_der y se clasifica la muestra como ESTABLE o INESTABLE.
Archivo de salida	Se genera resultado_pinza.txt con promedios, datos leídos y clasificación final.
Automatización con Bash	El script ejecuta el binario con sh, guarda el PID, espera, comprueba el proceso y lo finaliza si fuera necesario.

2. Explicación del flujo de trabajo

El trabajo sigue este orden: primero se lee el archivo de entrada, luego se guardan los datos en una matriz, se separan en arrays, se hacen los cálculos, se analiza la estabilidad del agarre y al final se genera el archivo de salida. Después, el script Bash automatiza la ejecución del programa.

1. Lectura de los registros del archivo datos_pinza.txt.
2. Almacenamiento temporal de las muestras en la matriz datos[100][3].
3. Separación de los valores en arrays independientes: ids, galga, fuerza_izq y fuerza_der.
4. Cálculo de sumas parciales y sumas globales para obtener los promedios finales.
5. Comparación entre fuerza izquierda y fuerza derecha para clasificar cada muestra.
6. Escritura del encabezado, los datos procesados, los promedios y la clasificación en resultado_pinza.txt.

3. Explicación del código C++ bloque a bloque

3.1 Bibliotecas y espacio de nombres

```
#include <iostream>
#include <fstream>
#include <cmath>
#include <string>
#include <cstdio>

using namespace std;
```

En este primer bloque se importan las librerías necesarias.

3.2 Inicio del programa y variables generales

```
int contador_total_registros = 0;
ifstream archivo_entrada("datos_pinza.txt");
ofstream archivo_salida("resultado_pinza.txt");
float suma_galga_total = 0, suma_izq_total = 0, suma_der_total = 0;
float max_fuerza = -1e9, min_fuerza = 1e9;
```

Aquí se crean el contador global de registros, los objetos de entrada y salida y varias variables acumuladoras. Además, se inicializan dos variables extra para guardar el valor máximo y mínimo de fuerza detectado durante el procesamiento.

3.3 Comprobación de errores al abrir archivos

```
if (!archivo_entrada.is_open()) {
    cout << "Error: no se pudo abrir el archivo" << endl;
    return 1;
} else if (!archivo_salida.is_open()) {
    cout << "Error: no se pudo abrir el archivo de salida" << endl;
    return 1;
}
```

Este bloque comprueba que los dos archivos principales se hayan abierto correctamente. Si falla la apertura del archivo de entrada o del archivo de salida, el programa termina inmediatamente con un código de error.

3.4 Escritura inicial y creación de archivos temporales

```
archivo_salida << "RESULTADOS DEL SISTEMA DE PINZA ROBOTICA" << endl;
ofstream temp_datos("temp_datos");
ofstream temp_clasificacion("temp_clasificacion");
```

Una vez validados los archivos, se escribe el encabezado del archivo de salida y se crean dos archivos temporales. El primero se usa para guardar los datos leídos y el segundo para almacenar la clasificación de cada muestra.

3.5 Bucle principal y declaración de la matriz y arrays

```
while (true)
{
    float datos[100][3];
    int ids[100];
    float galga[100], fuerza_izq[100], fuerza_der[100];
    string estado[100];
    int registrosLeidos = 0;
```

El programa trabaja por bloques de 100 registros. En cada vuelta del bucle principal se crean la matriz bidimensional y los arrays obligatorios que pide el enunciado. También se crea un array adicional llamado estado para guardar la clasificación ESTABLE o INESTABLE de cada muestra.

3.6 Lectura de datos y separación en arrays

```
while (registrosLeidos < 100 && archivo_entrada >> ids[registrosLeidos]
    >> datos[registrosLeidos][0] >> datos[registrosLeidos][1]
    >> datos[registrosLeidos][2]) {
    galga[registrosLeidos] = datos[registrosLeidos][0];
    fuerza_izq[registrosLeidos] = datos[registrosLeidos][1];
    fuerza_der[registrosLeidos] = datos[registrosLeidos][2];
    temp_datos << "ID:" << ids[registrosLeidos] << " Galga:" << galga[registrosLeidos]
        << " Izq:" << fuerza_izq[registrosLeidos]
        << " Der:" << fuerza_der[registrosLeidos] << endl;
    registrosLeidos++;
}
```

Este bloque realiza la lectura real del archivo. Cada línea se carga primero en la matriz datos y después se reparte en los arrays independientes. Al mismo tiempo, cada registro se escribe en el archivo temporal temp_datos para poder incorporarlo al archivo final más adelante.

3.7 Procesamiento, clasificación y máximos/mínimos

```
float suma_galga = 0, suma_izq = 0, suma_der = 0;
for (int i = 0; i < registrosLeidos; i++) {
    suma_galga += galga[i];
    suma_izq += fuerza_izq[i];
    suma_der += fuerza_der[i];
    float diferencia = fuerza_izq[i] - fuerza_der[i];
    if (abs(diferencia) > 0.15) {
        estado[i] = "INESTABLE";
    } else {
        estado[i] = "ESTABLE";
    }
    temp_clasificacion << ids[i] << " " << estado[i] << endl;
}
```

Aquí se calculan las sumas parciales del bloque y se evalúa la estabilidad de cada muestra. La condición usada sigue el criterio del enunciado: si la diferencia absoluta entre ambas fuerzas supera 0.15, la muestra pasa a ser INESTABLE. Además, cada clasificación se guarda en otro archivo temporal.

3.8 Comprobación del final del archivo y validación de lectura

```
if (registrosLeidos < 100) {  
    if (!archivo_entrada.eof()) {  
        cout << "Error: se produjo un error al leer el archivo" << endl;  
        return 1;  
    }  
}
```

Este bloque distingue entre dos situaciones distintas: que se haya llegado al final del archivo de forma normal o que se haya producido un error de lectura. Si todavía no se ha alcanzado EOF y no se han podido leer más registros, el programa informa del error y termina.

3.9 Cálculo final de promedios y escritura del resultado

```
archivo_salida << "\nPROMEDIOS:" << endl;  
archivo_salida << "Galga: " << (suma_galga_total /  
contador_total_registros) << endl;  
archivo_salida << "Fuerza Izq: " << (suma_izq_total /  
contador_total_registros) << endl;  
archivo_salida << "Fuerza Der: " << (suma_der_total /  
contador_total_registros) << endl;  
archivo_salida << "Valor maximo de fuerza: " << max_fuerza <<  
endl;  
archivo_salida << "Valor minimo de fuerza: " << min_fuerza <<  
endl;
```

```
PROMEDIOS:  
Galga: 0.236914  
Fuerza Izq: 1.72759  
Fuerza Der: 1.72693  
Valor maximo de fuerza: 2.2  
Valor minimo de fuerza: 1.36
```

Cuando ya se han procesado todos los bloques, el programa calcula los promedios globales y los escribe en el archivo de salida. También añade dos datos extra: la fuerza máxima y la fuerza mínima detectadas durante toda la ejecución.

3.10 Volcado de temporales, cierre y limpieza final

```
ifstream lectura_datos("temp_datos");  
while (getline(lectura_datos, linea)) {  
    archivo_salida << linea << endl;  
}  
...  
ifstream lectura_clasificacion("temp_clasificacion");  
while (getline(lectura_clasificacion, linea_clas)) {  
    archivo_salida << linea_clas << endl;  
}  
...  
remove("temp_datos");  
remove("temp_clasificacion");
```

En la parte final del programa se vuelcan al archivo de salida los datos y la clasificación guardados en los temporales. Después se cierran los archivos principales y se eliminan los temporales para no dejar ficheros auxiliares en la carpeta del proyecto. El programa termina mostrando un mensaje de confirmación.

```
Resultados guardados en 'resultado_pinza.txt'.  
El proceso finalizo correctamente.
```

4. Explicación del script Bash bloque a bloque

4.1 Cabecera del script

```
#!/usr/bin/env bash
```

La primera línea indica que el script debe ejecutarse con Bash. Es la forma estándar de hacer que el sistema use el intérprete correcto.

4.2 Ejecución del binario y guardado del PID

```
sh -c "./programaC" &  
PID=$!
```

En este bloque se ejecuta el binario compilado mediante sh y se lanza en segundo plano gracias al símbolo &. Después, con \$!, se guarda el identificador del proceso en la variable PID para poder controlarlo más adelante.

4.3 Espera a que el proceso termine

```
wait "$PID"
```

El comando wait hace que el script se quede esperando hasta que el proceso asociado a ese PID termine su ejecución.

4.4 Comprobación del estado del proceso

```
if kill -0 "$PID" 2>/dev/null; then
```

La instrucción kill -0 no mata el proceso. Solo comprueba si sigue existiendo. Si el proceso aún está activo, el script entra en la rama de finalización controlada.

4.5 Finalización controlada

```
kill -TERM "$PID"  
sleep 1
```

Primero se envía la señal SIGTERM para solicitar un cierre normal del proceso. Luego se espera un segundo para darle tiempo a terminar por sí mismo.

4.6 Cierre forzado si el proceso no responde

```
if kill -0 "$PID" 2>/dev/null; then  
    kill -KILL "$PID"  
fi
```

Si después de SIGTERM el proceso sigue vivo, se usa SIGKILL para forzar su finalización. Esta señal termina el proceso de forma inmediata.

4.7 Caso normal de finalización

```
else  
    echo "El proceso finalizó correctamente."  
fi
```

Si el proceso ya no está activo después de wait, el script muestra un mensaje indicando que la ejecución terminó correctamente. Con esto se cubre exactamente el comportamiento que exige el enunciado.

5. Análisis de resultados

El programa da como resultado un archivo del cual sacamos una pequeña muestra para facilitar su comprensión. El formato del archivo de resultado incluye un encabezado general, una sección con los promedios calculados, otra con los datos leídos y una última con la clasificación de cada muestra como estable o inestable. En general, los resultados permiten ver el comportamiento global del sistema y revisar ejemplos concretos de las mediciones analizadas.

5.1 Promedios globales

Magnitud	Valor
Galga	0.236914
Fuerza Izq	1.72759
Fuerza Der	1.72693
Valor máximo de fuerza	2.2
Valor mínimo de fuerza	1.36

```
PROMEDIOS:  
Galga: 0.236914  
Fuerza Izq: 1.72759  
Fuerza Der: 1.72693  
Valor maximo de fuerza: 2.2  
Valor minimo de fuerza: 1.36
```

5.2 Muestra de 5 registros del grupo DATOS

ID	Galga	Izq	Der
5118	0.24	1.796	1.752
6574	0.22	1.595	1.676

```
DATOS:  
ID:5118 Galga:0.24 Izq:1.796 Der:1.752  
ID:6574 Galga:0.22 Izq:1.595 Der:1.676
```

5.3 Muestra de 5 registros del grupo CLASIFICACIÓN

ID	Estado
5118	ESTABLE
6574	ESTABLE

```
22012 CLASIFICACION:  
22013 5118 ESTABLE  
22014 6574 ESTABLE
```